

GUIDA GRATUITA

Governare uno swarm di **agenti**

Le decisioni architetturali che contano prima del
codice

Omar Bortolato

Chief AI Officer · Imprenditore · Speaker

omarbortolato.it

AI pratica per chi vuole fare, non solo sapere.

PRIMA DI INIZIARE

Come leggere questa guida

Questa non è una guida a un prodotto. È la descrizione di un modo di strutturare un sistema di agenti, e delle ragioni per cui è fatto così.

Un agente è un programma che riceve una richiesta, ragiona con un modello linguistico e restituisce una risposta, eventualmente usando strumenti per farlo. Farne girare uno è semplice. Il problema di cui parla questa guida comincia quando ne hai dieci, ognuno con il proprio ambiente, il proprio budget e la possibilità di chiamare gli altri.

Ogni sezione risponde a tre domande, in quest'ordine. Qual è il problema strutturale. Quale decisione lo risolve, e per quale meccanismo. Cosa costa quella decisione, e in quali casi non conviene. Le trovi riassunte in fondo a ogni capitolo, in un blocco di tre righe.

La terza domanda non è un adempimento. Una guida che presenta solo i vantaggi di una scelta è materiale promozionale, e ti lascia senza gli strumenti per decidere diversamente. Il capitolo 9 è dedicato interamente a cosa si paga adottando questo modello, e riporta per intero l'argomento pubblico di chi sostiene che i sistemi multi-agente andrebbero evitati.

A chi serve

A chi costruisce automazioni con agenti e ha superato la fase del prompt singolo. Manager tecnici, imprenditori con competenze informatiche, project manager, professionisti che progettano sistemi. Serve sapere cos'è un'API. Non serve saper programmare.

Cosa non troverai

Codice da copiare, tutorial di installazione, confronti fra framework. L'architettura descritta è implementata in Python, ma le decisioni di cui parlo non dipendono dal linguaggio né dalla libreria di orchestrazione che usi. Il capitolo 10 le riformula come domande a cui rispondere sul tuo stack, qualunque esso sia.

Troverai invece riferimenti a principi consolidati in altri campi dell'informatica, alcuni con cinquant'anni di storia. Non è erudizione: quasi tutti i problemi di un sistema di agenti sono problemi di sistemi distribuiti travestiti, e sono già stati risolti almeno una volta.

SUI RIFERIMENTI VOLATILI

Nomi di modelli, prezzi, limiti di piattaforma e versioni cambiano ogni pochi mesi. Dove un riferimento del genere è indispensabile, sta dentro un riquadro tratteggiato marcato come aggiornabile. Il resto del testo punta a principi che non scadono.

INDICE

Cosa c'è dentro

1	Il salto da uno a dieci	4
	Perché orchestrare dieci agenti è un problema di natura diversa. La tassonomia dei pattern di orchestrazione.	
2	Il control-plane	8
	Le quattro responsabilità che non sono delegabili, e perché stanno tutte nello stesso punto.	
3	Il contratto minimo	11
	Perché un agente è una cartella e non una riga in un registro.	
4	La topologia	15
	I rank, la regola di delega, il DAG come proprietà dell'ordinamento e non come controllo.	
5	Il gating	21
	Le regole valgono nel punto di passaggio obbligato, non in un documento che nessuno legge.	
6	Il ciclo di una richiesta	25
	I controlli in sequenza, il ragionamento dietro l'ordine, e perché un chiarimento non è un errore.	
7	Agenti e capacità	28
	Quando un'unità di lavoro merita un'identità, e quando basta una funzione invocabile.	
8	La spesa come vincolo di governance	31
	I livelli di cache, cosa saltano, e la regola che nessuno di essi può violare.	
9	Limiti e trade-off dell'architettura	34
	Cosa si paga adottando questo modello, e l'argomento di chi sostiene di non adottarlo.	
10	Le decisioni che devi prendere tu	37
	Dieci scelte architetturali con i criteri per decidere, applicabili a stack diversi.	
11	Fonti e letture	39
	Bibliografia ragionata, con data di consultazione.	

CAPITOLO 1

Il salto da uno a dieci

Far girare un agente è facile: un processo, un prompt, una risposta. Quello che non è facile è farne girare dieci senza che il sistema diventi impossibile da capire.

La differenza non è di scala, è di natura. Un agente solo è un programma: ha un ingresso, una logica e un'uscita, e quando sbaglia lo vedi. Dieci agenti che possono chiamarsi fra loro sono un sistema distribuito, e i sistemi distribuiti falliscono in modi che non assomigliano a un bug. Falliscono lentamente, in silenzio, e spesso il primo segnale è una fattura.

Vale la pena essere precisi sui modi in cui succede, perché sono quattro e sono diversi fra loro. Ognuno richiede una difesa diversa, e confonderli porta a costruire la difesa sbagliata.

Il ciclo

Un agente ne chiama un altro, che a un certo punto richiama il primo. Nessuno dei due sta facendo qualcosa di sbagliato: ciascuno delega a chi sembra più competente su quella parte del problema. Il ciclo emerge dalla composizione di decisioni locali ragionevoli, e non è visibile guardando il codice di nessuno dei due.

La versione costosa di questo problema non è il ciclo infinito, che almeno si nota. È il ciclo che gira quattro volte prima di risolversi, su una richiesta che ne arriva mille al giorno. Nessun allarme scatta, il sistema funziona, e il costo è quadruplicato in modo permanente.

Il fan-out

Un agente decide che il compito si divide in cinque parti e ne delega cinque. Ognuno dei cinque fa lo stesso. Al terzo livello sono centoventicinque processi, e nessuno ha mai preso la decisione di avviarne centoventicinque. La ramificazione è il comportamento normale di ogni singolo nodo: è la sua composizione ad essere esplosiva.

Anthropic ha misurato l'ordine di grandezza in un resoconto pubblico sul proprio sistema multi-agente di ricerca: gli agenti consumano circa quattro volte i token di una conversazione, e i sistemi multi-agente circa quindici volte.² Il numero esatto conta poco e invecchierà. Conta che il moltiplicatore sia una proprietà dell'architettura, non dell'implementazione.

La degradazione del contesto

Ogni passaggio di consegne fra agenti comporta un riassunto. Chi delega non trasferisce tutto quello che sa, trasferisce quello che ritiene rilevante. È una compressione con perdita, e la perdita si accumula lungo la catena.

C'è anche un effetto più sottile, e misurato: le prestazioni dei modelli linguistici degradano quando l'informazione rilevante si trova in mezzo a un contesto lungo, invece che all'inizio o alla fine.⁵ Un agente che riceve un contesto accumulato da tre passaggi precedenti non è nella stessa condizione di uno che riceve la domanda originale.

La propagazione dell'errore

Un errore commesso al primo livello non viene corretto ai livelli successivi: viene assunto come dato. Chi sta sotto non ha modo di sapere che la premessa è sbagliata, perché gli arriva come richiesta, non come ipotesi. Il sistema produce con grande efficienza una risposta coerente a una domanda sbagliata.

Su questo esiste ora un lavoro sistematico. Un gruppo di ricerca di Berkeley ha analizzato oltre milleseicento tracce di esecuzione raccolte su sette framework multi-agente diversi, e ne ha ricavato una tassonomia di quattordici modi di fallire, raggruppati in tre categorie: problemi di progettazione del sistema, disallineamento fra agenti, e verifica del compito.³

La conclusione che conta, per chi progetta, è questa: molti fallimenti non dipendono dalla qualità del modello ma dalla struttura del sistema. Cambiare modello non li risolve. È una buona notizia, perché la struttura è l'unica cosa su cui hai davvero controllo.

IL PUNTO DI QUESTO CAPITOLO

Nessuno dei quattro problemi si risolve scrivendo agenti migliori. Sono tutti proprietà delle relazioni fra agenti, non degli agenti. Le decisioni che li affrontano vanno prese sulla struttura, e vanno prese prima, perché dopo si tratta di riscrivere.

Cinque modi di orchestrare

Prima di guardare una soluzione conviene sapere quali forme esistono. Ogni pattern risolve un problema diverso lasciandone aperto un altro.

Il **supervisor** mette un agente al centro: riceve la richiesta, decide chi deve lavorare, raccoglie i risultati. La terminazione è garantita perché il controllo torna sempre allo stesso posto, e il centro diventa il collo di bottiglia.

Il **gerarchico** generalizza il supervisor su più livelli: supervisor di supervisor.⁶ Scala in ampiezza, e allunga la catena lungo cui un errore può propagarsi senza essere notato.

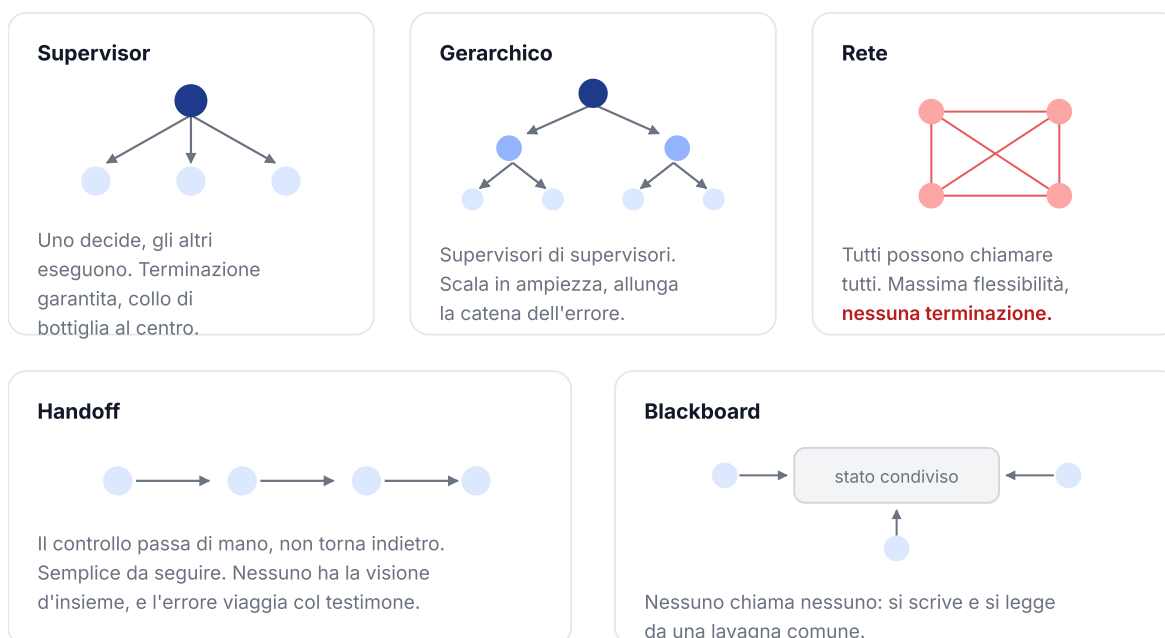
La **rete** lascia che ogni agente chiami ogni altro. È la forma più flessibile e l'unica senza garanzia di terminazione: la struttura ammette i cicli, quindi devi rilevarli mentre accadono.

L'**handoff** passa il controllo in avanti, senza ritorno: ogni agente completa la sua parte e consegna al successivo. È facile da tracciare, ma nessuno ha la visione d'insieme, e un errore introdotto presto viaggia con il testimone fino in fondo.

La **blackboard** elimina la chiamata diretta: gli agenti scrivono e leggono da uno stato condiviso, e ognuno interviene quando riconosce una situazione su cui sa agire. Viene dall'intelligenza artificiale degli anni Ottanta⁷ e risolve bene il problema del contesto condiviso, spostando la difficoltà sul controllo di chi agisce quando.

GOVERNARE UNO SWARM DI AGENTI · FIG. 8

Cinque modi di orchestrare



DOVE SI COLLOCA L'ARCHITETTURA DESCRITTA

Gerarchica con un vincolo in più: i livelli sono ordinati, e la delega può solo scendere. È la variante che rinuncia alla flessibilità della rete in cambio di una garanzia che nessun controllo a runtime può dare.

Nessuno dei cinque è migliore in assoluto. Cambia cosa garantiscono, e quindi cosa devi controllare tu.

Cinque modi di orchestrare. Cambia cosa garantiscono per costruzione, e quindi cosa resta a carico tuo.

Dove si colloca l'architettura descritta

L'architettura di cui parla questa guida è gerarchica, con un vincolo in più: i livelli sono ordinati, e la delega può solo scendere. Non è una variante di dettaglio. È la scelta che trasforma l'assenza di cicli da controllo da eseguire a proprietà da non dover verificare, e il capitolo 4 è dedicato interamente a questo.

Il prezzo è dichiarato fin d'ora: si rinuncia alla flessibilità della rete. Due agenti dello stesso livello non possono collaborare direttamente, e ogni collaborazione trasversale richiede una struttura esplicita. Se il tuo problema ha bisogno di negoziazione fra pari, questa non è l'architettura giusta, e non c'è modo di aggirarlo senza smontare la garanzia.

Vale la pena dire anche cosa questa scelta non risolve. L'ordinamento garantisce che il sistema termini e che nessuno chiami chi non deve. Non dice niente sulla qualità del passaggio di consegne, e quindi non affronta la degradazione del contesto né la propagazione dell'errore. Quelli restano problemi aperti, e il capitolo 9 li riprende.

PRIMA DI AGGIUNGERE AGENTI

La raccomandazione più utile della documentazione tecnica di Anthropic sul tema è di aggiungere complessità solo quando migliora il risultato in modo dimostrabile, e di ricordare che ottimizzare una singola chiamata con recupero di informazioni ed esempi in contesto è spesso sufficiente.¹ Un sistema di dieci agenti che risolve un problema da un agente è un sistema peggiore, non più avanzato.

IL PROBLEMA	Da un certo numero di agenti in poi, il comportamento del sistema smette di essere deducibile dal codice dei singoli agenti: cicli, fan-out, degradazione del contesto ed errori che si propagano nascono dalle relazioni, non dai componenti.
LA DECISIONE	Scegliere consapevolmente un pattern di orchestrazione, sapendo cosa garantisce per costruzione e cosa lascia da verificare a runtime, invece di lasciare che la topologia emerga da chi chiama chi.
COSA COSTA	Ogni pattern rinuncia a qualcosa. Scegliere in anticipo significa rinunciare in anticipo, quando ancora non sai con certezza di cosa avrai bisogno.

CAPITOLO 2

Il control-plane

Un control-plane è il componente che decide, e che non esegue. Nei sistemi di rete è il piano che stabilisce le rotte, distinto dal piano che fa passare i pacchetti. Qui è il punto da cui si entra, e l'unico posto in cui le regole possono essere davvero applicate.

La formulazione precisa arriva dalla sicurezza informatica e ha cinquant'anni. Saltzer e Schroeder, nel 1975, la chiamano *complete mediation*: ogni accesso a ogni oggetto deve essere verificato per autorità.¹⁰ Una regola che vale solo su alcuni percorsi di accesso non è una regola, è una convenzione.

Da qui discende la struttura. Se esistono due modi di far partire un agente, le regole valgono su uno solo dei due, e il secondo diventa la scorciatoia che qualcuno userà quando ha fretta. Il control-plane non è un punto di ingresso privilegiato: è l'unico.

Le quattro responsabilità

Il control-plane possiede quattro cose che nessun agente deve possedere da sé. Non è una lista arbitraria: sono le quattro cose che, se lasciate al singolo agente, funzionano finché qualcuno non se le dimentica.

Il ciclo di vita del processo. Chi avvia un agente, con quale ambiente, con quale timeout, e chi lo termina. Se ogni agente gestisce il proprio avvio, ogni agente ha una versione leggermente diversa di cosa significhi partire, e nessuno sa quale sia quella giusta.

Il gating. La verifica di conformità all'ingresso, con l'esito e il motivo. È il capitolo 5. Qui basti dire che un controllo che l'agente fa su se stesso non è un controllo: è una dichiarazione.

Il protocollo delle risposte non semplici. Un agente può rispondere, ma può anche chiedere un chiarimento o emettere la risposta a pezzi. Sono forme diverse, e chi le riceve deve poterle distinguere senza interpretare il testo. Se la distinzione sta dentro l'agente, ogni agente la implementa a modo suo.

Il limite di profondità della ricorsione. Quanto in là può arrivare una catena di deleghe. Questo è il caso in cui la ragione per non delegare al singolo agente è più evidente: un agente che si dimentica di contare è un agente che non ferma niente, e il costo ricade su tutti.

Perché non sono delegabili

C'è un argomento comune alle quattro. Sono tutte responsabilità il cui fallimento non si manifesta dove è stato causato. Un agente che dimentica il limite di profondità non produce un errore: produce lavoro. Un agente che gestisce male il proprio ciclo di vita non si accorge di niente, e il problema lo scopre chi legge i log del sistema tre giorni dopo.

Questa è la differenza fra una regola di sviluppo e un vincolo architetturale. La prima dipende dalla disciplina di chi scrive, e la disciplina è una risorsa che si esaurisce sotto pressione. Il secondo vale anche quando nessuno ci sta pensando.

Il parallelo con l'*admission control*

Il modello non è nuovo. Kubernetes, il sistema di orchestrazione di container più diffuso, ha un meccanismo chiamato *admission controller*: ogni richiesta di creare o modificare una risorsa passa da una catena di controlli prima di essere accettata.¹¹ Alcuni modificano la richiesta per portarla allo standard, altri la accettano o la rifiutano. Se un controllo rifiuta, l'intera richiesta viene rifiutata.

Le due proprietà interessanti sono che i controlli stanno fuori dall'oggetto controllato, e che non esistono percorsi che li aggirano. È esattamente la struttura del *gating* descritto nel capitolo 5, applicata a un dominio diverso vent'anni prima.

Lo stesso vale per la *policy-as-code*: scrivere le regole di autorizzazione come codice valutabile e versionato, separato dall'applicazione che le subisce.¹² La ragione per cui questi sistemi esistono è che le regole scritte in un documento di convenzioni non vengono applicate, vengono ricordate. E ricordate male.

Cosa non deve possedere nessun agente

Il complemento della lista precedente è altrettanto importante. Tre cose non devono mai stare dentro il codice di un agente, e sono tutte cose che tecnicamente potrebbero starci.

Le credenziali dei provider. Una chiave nel sorgente di un agente è una chiave che finisce nella cronologia del repository, che viaggia in ogni copia, e che nessuno sa più dove sia quando va ruotata. Il principio è vecchio quanto il resto: la configurazione, e in particolare i segreti, sta nell'ambiente e non nel codice.¹⁴

L'accesso diretto al provider. Anche senza chiave nel sorgente, un agente che chiama direttamente la libreria di un fornitore di modelli aggira il varco attraverso cui passa la contabilità. Non è un problema di sicurezza, è un problema di governance: quello che non passa dal varco non è misurato, e quello che non è misurato non è limitabile.

La decisione su chi può chiamarlo. Un agente non è la sede giusta per decidere chi ha il diritto di invocarlo, per la stessa ragione per cui una porta non decide chi ha la chiave. Questa decisione appartiene alla topologia, ed è il capitolo 4.

ATTENZIONE AL PUNTO DEBOLE

Un control-plane concentra tutto in un punto, e quindi concentra anche il rischio. Se cade, non gira niente. Se ha un difetto nella logica di autorizzazione, il difetto vale per tutto il sistema contemporaneamente. È il rovescio esatto della proprietà che lo rende utile, e va messo in conto: la concentrazione paga in coerenza e si paga in fragilità.

IL PROBLEMA	Le regole che valgono solo su alcuni percorsi di accesso non sono regole. Le responsabilità il cui fallimento è silenzioso non possono essere lasciate a chi ha interesse a non esercitarle.
LA DECISIONE	Un unico punto di ingresso che possiede ciclo di vita, gating, protocollo e limite di profondità. Nessun agente possiede credenziali, accesso diretto al provider o giudizio su chi può chiamarlo.
COSA COSTA	Un punto di fallimento unico e un collo di bottiglia sotto carico. In una fase di esplorazione rallenta, perché ogni prova passa comunque dal controllo.

CAPITOLO 3

Il contratto minimo

Un agente è una cartella. Non esiste un registro da aggiornare a mano: il control-plane scopre gli agenti leggendo il disco.

La frase che giustifica la scelta è breve: un registro che diverge dal disco è un registro che mente, e prima o poi qualcuno ci costruisce sopra una decisione. Vale la pena aprirla, perché è uno dei punti in cui la pratica dei sistemi di agenti può imparare qualcosa di già stabilito altrove.

Stato dichiarato e stato osservato

Un registro è una dichiarazione: qualcuno ha scritto che esistono sette agenti con queste caratteristiche. Il disco è un'osservazione: queste sono le cartelle che ci sono adesso. Finché coincidono, la differenza è filosofica. Divergono al primo momento in cui qualcuno aggiunge una cartella e dimentica di aggiornare il registro, e da quel momento il sistema ha due verità.

È il problema che in ambito infrastrutturale si chiama deriva della configurazione, e la risposta consolidata è il ciclo di riconciliazione: un processo che confronta continuamente lo stato desiderato con lo stato osservato e agisce fino a farli coincidere.¹³ In Kubernetes ogni oggetto porta con sé entrambi, la specifica e lo stato, ed è il controller a ridurre la distanza.

La scoperta da disco è la versione minima dello stesso principio: se lo stato è sempre derivato dalla realtà, la distanza è zero per costruzione e non serve nessun ciclo. Si rinuncia a qualcosa, e lo vediamo alla fine del capitolo.

C'è un criterio generale sotto, e vale ben oltre gli agenti. Ogni volta che una lista deve essere tenuta allineata a mano con qualcosa che esiste già, quella lista è un debito. La domanda giusta non è come ricordarsi di aggiornarla, ma se sia possibile derivarla.

Cosa contiene la cartella

Un file è obbligatorio, tre sono facoltativi. La proporzione è deliberata: chi prototipa deve poter scrivere una cartella e vederla funzionare, non compilare un modulo.

GOVERNARE UNO SWARM DI AGENTI · FIG. 5

Il contratto dell'agente



Un file obbligatorio, tre facoltativi. Chi prototipa scrive una cartella; chi governa legge sempre la stessa struttura.

Il contratto dell'agente. Un solo file obbligatorio. Tutto il resto viene dedotto, e la deduzione è dichiarata invece che silenziosa.

La persona sta scritta in un posto solo

La persona di un agente è la descrizione del ruolo che assume quando ragiona: chi è, cosa sa, come risponde. È il testo che finisce nell'istruzione di sistema del modello. In questa architettura sta in un file dedicato, e il codice la importa invece di ripeterla.

La ragione è la regola più antica della progettazione modulare: ogni modulo nasconde una decisione, e le decisioni che possono cambiare vanno isolate in un posto solo.¹⁷ Parnas la formulò nel 1972 per i moduli software, e vale identica qui. Se la persona compare in due posti, prima o poi cambia in uno solo, e il sistema ha due comportamenti diversi a seconda del percorso.

Il caso concreto è il più banale possibile: qualcuno affina il tono di un agente nel file della persona, e non si accorge che una versione più vecchia è incollata dentro il codice. L'agente risponde con il tono vecchio. Nessun errore, nessun log, nessun modo di accorgersene se non leggendo.

I tag stanno solo nell'header

I tag di governance sono le etichette che dichiarano la posizione dell'agente nella topologia e le sue capacità di protocollo. Si leggono solo nel blocco di commenti iniziale del file di ingresso. Citarli più in basso, in una docstring o in un commento, non attiva niente.

È una restrizione che a prima vista sembra pignoleria e invece è la parte più importante del contratto. Un sistema che si configura leggendo il proprio codice non deve poter essere configurato per sbaglio. Se bastasse nominare un tag ovunque nel file, un esempio dentro un commento, una riga di documentazione o un test che descrive un caso diventerebbero configurazione attiva.

La stessa logica vale al contrario: siccome il posto è uno solo ed è in cima, chiunque apra il file vede la governance dell'agente nelle prime righe, senza cercarla.

IL CRITERIO GENERALE

Quando una dichiarazione ha effetti sul comportamento del sistema, il posto in cui si scrive deve essere unico e riconoscibile a colpo d'occhio. Ogni posto in più è un posto in cui può finire per caso.

Cosa arriva dall'import

L'ultima parte del contratto è quella che non si scrive. Cache, memoria condivisa, guardrail, telemetria e formato della risposta arrivano dall'importazione del kernel comune: non sono codice che l'autore dell'agente deve produrre.

La frase che spiega la scelta merita di essere isolata: se risparmiare token fosse qualcosa da ricordarsi, prima o poi non ce lo ricorderemmo. Vale per tutte e cinque le cose in elenco. Una funzionalità trasversale che dipende dalla diligenza di chi scrive il singolo componente è una funzionalità che hai a metà, e non sai quale metà.

C'è un effetto collaterale utile. Siccome l'agente minimo conforme è corto, il costo di scriverne uno nuovo è basso, e quindi il contratto non viene percepito come un ostacolo. Un contratto che costa poco viene rispettato; uno che costa molto viene aggirato, e a quel punto non descrive più niente.

Cosa costa la scoperta da disco

Tre cose, e vanno dette. La prima: il disco diventa l'interfaccia. Spostare una cartella, rinominarla o lasciarne una a metà ha effetti immediati sul sistema, senza nessun passaggio intenzionale. Un registro, con tutti i suoi difetti, richiede un gesto esplicito per cambiare la realtà.

La seconda: non c'è storia. Un registro versionato dice chi ha aggiunto cosa e quando. Il disco dice solo com'è adesso, e per sapere com'era ieri devi guardare altrove.

La terza: la scoperta costa a ogni lettura, e su un numero grande di agenti diventa un costo da gestire con una cache, che reintroduce esattamente il problema della divergenza che si voleva evitare. È il punto in cui la scelta va rivista, non prima.

IL PROBLEMA	Uno stato dichiarato a mano diverge dalla realtà al primo momento di distrazione, e da lì il sistema ha due verità. Le informazioni duplicate cambiano in un posto solo.
LA DECISIONE	Derivare lo stato dall'osservazione invece che dalla dichiarazione. Un contratto minimo con un solo file obbligatorio, la persona in un posto solo, i tag in una posizione unica e riconoscibile.
COSA COSTA	Il filesystem diventa un'interfaccia, con la fragilità che comporta. Si perde la storia dei cambiamenti. Su molti agenti la scoperta va messa in cache, e la cache riporta il problema di partenza.

CAPITOLO 4

La topologia

Ogni agente dichiara dove sta nel grafo di delega. Il tag non è un'etichetta descrittiva: è la sua posizione, e determina chi può chiamarlo.

Questo è il capitolo centrale, e la ragione è che la decisione che descrive è l'unica del documento che produce una garanzia invece di una difesa. Le altre riducono la probabilità che qualcosa vada storto. Questa rende una classe di problemi non rappresentabile.

Quattro livelli

Il grafo ha quattro posizioni, e ognuna dice due cose: cosa fa l'agente, e chi può raggiungerlo.

Rank 0, entrata. Da dove entra una richiesta nel sistema. Non è mai il bersaglio di un altro agente: è il punto in cui il lavoro comincia, non uno a cui si delega.

Rank 1, coordinatore. Orchestra il proprio dominio. È raggiungibile solo su chiamata esplicita, mai per instradamento automatico. Qui stanno i ruoli di responsabilità verticale, quelli che nell'organigramma di un'azienda chiameresti capi funzione.

Rank 2, ponte. Attraversa i domini. È raggiungibile solo da un dominio diverso dal proprio: esiste per far comunicare due verticali senza che i loro responsabili si parlino direttamente.

Rank 3, foglia. Risponde e basta. Una foglia che delega non è una foglia: è il livello in cui il lavoro si esaurisce, e la sua definizione coincide con questa proprietà.

```
delega ammessa  $\iff$  rank(chiamante) < rank(bersaglio)
```

Una sola disuguaglianza. Tutto il resto di questo capitolo è la spiegazione di cosa discende da qui, e di quanto sia diverso da una regola che si controlla.

Perché non ci possono essere cicli

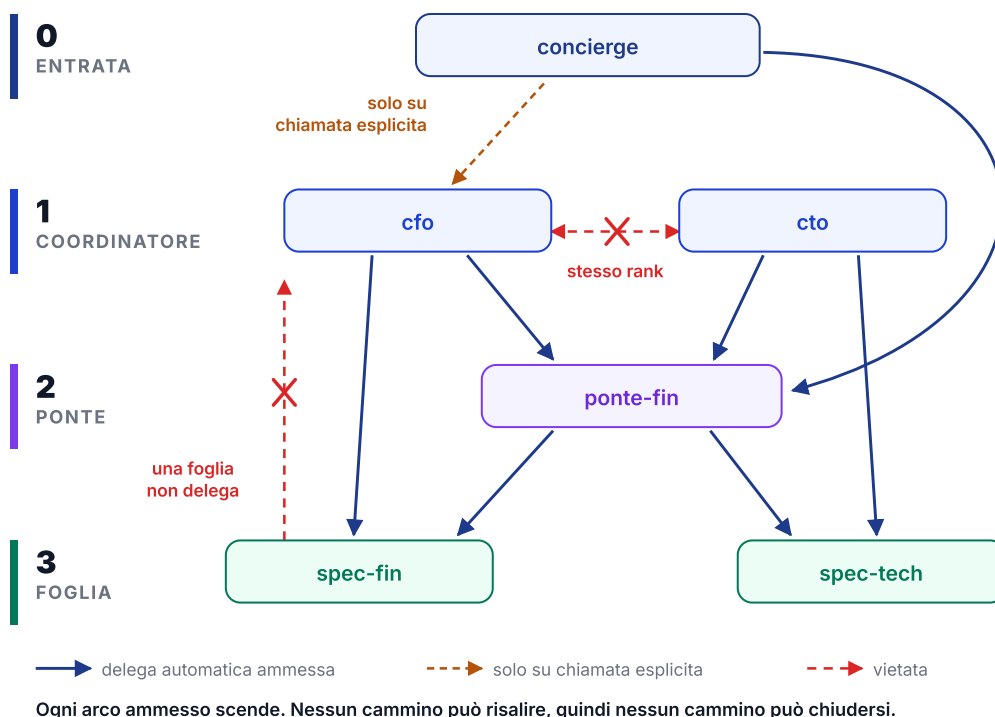
La relazione "minore di" sui numeri è un ordine stretto. Ha due proprietà che qui contano: è irreflessiva, cioè nessun elemento è minore di se stesso, ed è transitiva, cioè se il primo è minore del secondo e il secondo del terzo, allora il primo è minore del terzo.

Da queste due proprietà segue tutto. Immagina un cammino di deleghe che parta da un agente e ci ritorni. Ogni passo del cammino aumenta il rank, per la regola. Per transitività, il rank di partenza sarebbe minore del rank di arrivo. Ma partenza e arrivo sono lo stesso agente, quindi servirebbe che il suo rank fosse minore di se stesso. Che è escluso dall'irriflessività.

Il cammino non esiste. Non è raro, non è improbabile, non è intercettato: non è rappresentabile nel sistema di regole. Il grafo di delega è un grafo aciclico diretto, un DAG, per costruzione e non per verifica.

Un DAG è precisamente un grafo su cui esiste un ordinamento topologico: una disposizione lineare dei nodi in cui ogni arco punta sempre in avanti.⁸ Qui l'ordinamento topologico non va calcolato, perché è dato: sono i rank. Il grafo non viene ordinato dopo essere stato costruito, viene costruito dentro un ordine.

GOVERNARE UNO SWARM DI AGENTI · FIG. 1



Il grafo di delega, quattro livelli. Le frecce ammesse scendono sempre. Le due vietate mostrano i due modi tipici di sbagliare: chiamare un pari e risalire.

Perché il rilevamento a runtime non basta

L'alternativa naturale è lasciare che i cicli siano possibili e accorgersene mentre accadono: si tiene traccia della catena di chiamate e si interrompe quando un nome si ripete, oppure si conta la profondità e ci si ferma a una soglia. È la soluzione che quasi tutti adottano, ed è ragionevole. Ha tre debolezze, e vanno capite prima di scegliere.

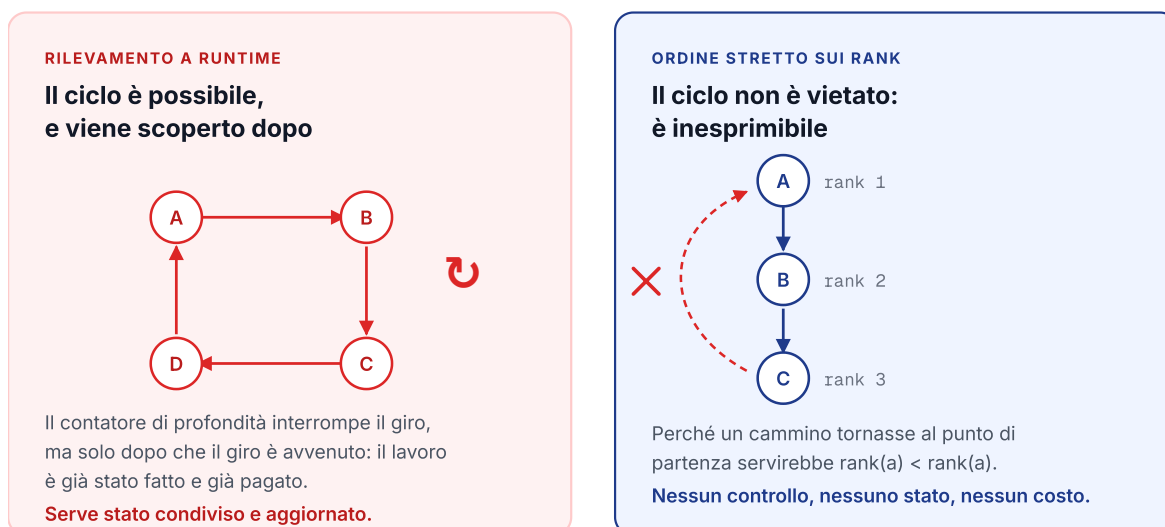
Interviene dopo. Nel momento in cui il rilevamento scatta, il giro è già avvenuto almeno una volta. Il lavoro è stato fatto, i token sono stati spesi, e in un sistema che gestisce molte richieste il costo di ciò che viene interrotto è comunque sostenuto.

Richiede stato condiviso e corretto. Per sapere che si sta ripetendo un nome bisogna sapere chi è già stato chiamato, e questa informazione deve viaggiare con la richiesta o vivere in un posto che tutti leggono. È un pezzo di infrastruttura in più, che può perdersi, disallinearsi o essere aggirato da chi chiama fuori dal percorso previsto.

Confonde due problemi. Un limite di profondità ferma sia il ciclo sia la catena lunga ma legittima, e non sa distinguerli. Se lo alzi per non tagliare il lavoro buono, lasci correre più a lungo quello cattivo. Non esiste una soglia giusta perché la soglia sta misurando la cosa sbagliata.

Il parallelo esatto viene dai sistemi operativi. Il deadlock, la situazione in cui più processi si bloccano a vicenda aspettando risorse, ha quattro condizioni necessarie, e una di queste è l'attesa circolare.⁹ Si può rilevare il deadlock quando è accaduto, oppure si può imporre un ordine totale sulle risorse e pretendere che vengano acquisite in quell'ordine: a quel punto l'attesa circolare è impossibile e il deadlock non accade.¹⁵ È la stessa mossa, su un problema diverso, e ha sessant'anni.

GOVERNARE UNO SWARM DI AGENTI - FIG. 9

Cicli rilevati contro cicli impossibili

La differenza non è di severità. È di categoria: nel primo caso il sistema si difende da una cosa che può accadere, nel secondo quella cosa non è rappresentabile.

È lo stesso meccanismo con cui si previene il deadlock ordinando le risorse, invece di rilevarlo quando si è già verificato.

Cicli rilevati contro cicli impossibili. La differenza non è di severità del controllo: è che nel secondo caso non c'è niente da controllare.

La regola dei pari

Dalla disuguaglianza discende una conseguenza che sorprende chi la incontra la prima volta. Due coordinatori hanno lo stesso rank, quindi la disuguaglianza è falsa in entrambe le direzioni, quindi non possono chiamarsi fra loro. Non perché sia vietato in una tabella: perché la formula non lo ammette.

L'equivalente organizzativo è immediato, e vale la pena guardarlo perché rende evidente che non è una limitazione tecnica. Il responsabile amministrativo non comanda il responsabile tecnico. Entrambi dirigono chi sta sotto di loro. Quando devono coordinarsi, o sale la decisione a chi sta sopra, oppure si crea una figura trasversale che appartiene a nessuno dei due domini.

Il rank 2, il ponte, è esattamente quella figura trasversale. Esiste perché la collaborazione fra pari è un bisogno reale, e la risposta è darle una struttura invece di permettere una chiamata diretta.

La funzione che decide

L'ordine sui rank è il fondamento, ma non è l'intera regola. Sopra di esso stanno alcuni vincoli di governance, e l'ordine in cui vengono valutati è significativo.

Prima si escludono i casi degeneri: un agente non chiama se stesso, e un agente marcato come privato non è raggiungibile per instradamento automatico da nessuno. Poi si esclude l'entrata come bersaglio, perché il punto di ingresso non è un destinatario. Solo a questo punto si applica l'ordine topologico, ed è qui che i cicli muoiono.

Superato l'ordine, restano tre casi. Il ponte è raggiungibile solo se il chiamante appartiene a un dominio diverso dal suo. Il coordinatore non è mai raggiungibile per instradamento automatico, solo su chiamata esplicita. La foglia è raggiungibile.

C'è un dettaglio che vale l'intero capitolo: l'aciccità non è affidata alle regole di dominio. Le regole di dominio sono simmetriche, cioè se A e B stanno in domini diversi la condizione vale in entrambe le direzioni, e una condizione simmetrica non può escludere un ciclo di lunghezza due. La garanzia viene solo dall'ordine sui rank, e le regole di dominio stanno sopra di esso senza mai sostituirlo.

Un'ultima cosa. Una richiesta che arriva dall'esterno, da una persona e non da un agente, si comporta come se avesse rank meno uno: sta prima di tutti e può raggiungere qualsiasi posizione che le altre regole non escludano. È coerente, perché una persona non è un nodo del grafo e non può quindi essere il punto di ritorno di un ciclo.

GOVERNARE UNO SWARM DI AGENTI · FIG. 2

Chi può chiamare chi

DA \ VERSO	CONCIERGE	CFO	CTO	PONTE-FIN	SPECIALISTA
conciERGE 0	*	*	*	✓	✓
cfo 1	*	*	*	✓	✓
cto 1	*	*	*	✓	✓
ponte-fin 2	*	*	*	*	✓
specialista 3	*	*	*	*	*
tutti gli altri	*	*	*	*	*

nessuno è mai bersaglio automatico di rank 0 o 1

La metà vuota non è un buco: è la forma che l'ordine impone al grafo.

il lavoro si accumula in fondo

Chi può chiamare chi. La matrice non è disegnata a mano: è calcolata con la stessa funzione che il sistema applica a runtime. Non descrive il grafo, è il grafo.

Documentazione che non può mentire

Vale la pena fermarsi su quest'ultimo punto, perché è una tecnica riutilizzabile ovunque. La matrice che hai appena visto non è una tabella scritta da qualcuno che ha guardato il codice. È generata invocando la stessa funzione che il sistema usa per decidere, su tutte le coppie possibili.

La conseguenza è che la documentazione non può divergere dal comportamento. Se qualcuno cambia la regola e dimentica di aggiornare il documento, il documento cambia da solo. Se il documento dice una cosa e il sistema ne fa un'altra, è un bug del generatore, non una dimenticanza.

È lo stesso principio del capitolo precedente applicato alla documentazione invece che all'inventario: quello che può essere derivato non va dichiarato.

IL COSTO VERO DELLA GERARCHIA

Un ordinamento rigido rende alcune cose impossibili, e alcune di quelle cose ti serviranno. Due specialisti che dovrebbero confrontarsi devono passare da chi sta sopra, con un giro in più e una perdita di contesto. Promuovere una foglia a coordinatore è una decisione deliberata, non un aggiustamento. Se il tuo dominio richiede negoziazione continua fra pari, stai combattendo contro l'architettura invece di usarla.

IL PROBLEMA

In un grafo di delega libero i cicli sono possibili, e ogni difesa contro di essi interviene dopo che il costo è già stato sostenuto, richiede stato condiviso e non sa distinguere un ciclo da una catena lunga legittima.

LA DECISIONE

Assegnare a ogni agente una posizione in un ordine stretto e ammettere la delega solo verso posizioni successive. L'aciclicità diventa una proprietà dell'ordinamento, non un controllo da eseguire.

COSA COSTA

I pari non collaborano direttamente e serve una figura ponte. La gerarchia va decisa presto e cambiarla è un riassetto. Non c'è modo di fare un'eccezione senza perdere la garanzia, perché la garanzia è l'assenza di eccezioni.

CAPITOLO 5

Il gating

Chi prototipa non deve conoscere il contratto in anticipo. Sperimenta, e all'ingresso il sistema normalizza ciò che è normalizzabile e trattiene il resto, col motivo.

Il gating è il controllo di conformità che un agente attraversa prima di poter girare. La tesi del capitolo è che le regole di sviluppo scritte in un documento non vengono applicate: vengono ricordate, e ricordate male. L'unico posto in cui una regola vale davvero è il punto di passaggio obbligato.

La proprietà che rende il gating diverso da una linea guida è che non esiste la via di mezzo. Un agente è conforme e gira, oppure è trattenuto e non gira. Non esiste lo stato "gira ma chissà cosa fa", che è precisamente lo stato in cui si trova la maggior parte dei sistemi costruiti in fretta.

Tre esiti, e nessun quarto

Il primo esito è la conformazione. Qualcosa manca ed è deducibile da quello che c'è: viene dedotto, l'agente gira, e la deduzione è scritta. Non è indulgenza, è il riconoscimento che obbligare a compilare campi che il sistema può ricavare da solo produce campi compilati male.

Il secondo è l'attesa. L'agente è conforme ma manca un presupposto per eseguirlo, tipicamente l'ambiente delle dipendenze. Non è una violazione, è un lavoro non ancora fatto, e merita una risposta diversa da un rifiuto.

Il terzo è la quarantena. Qualcosa non torna e non è deducibile: l'agente viene trattenuto, e il motivo viene scritto accanto. La parte importante è la seconda: un rifiuto senza motivo costringe chi lo riceve a indovinare, e chi indovina finisce per cambiare cose a caso.

GOVERNARE UNO SWARM DI AGENTI · FIG. 4

Conformato, in attesa, quarantena

SITUAZIONE	ESITO	PERCHÉ
manifest assente	CONFORMATO	dedotto da AGENT.md, non cambia il comportamento dell'agente
dominio non dichiarato	CONFORMATO	classificato dal testo, senza modello: deterministico e quindi ispezionabile
descrizione assente	CONFORMATO	prima riga di prosa della persona
#STREAM + #CLARIFY	QUARANTENA	protocolli incompatibili: il chiarimento finirebbe nello stream come testo
nessun tag topologico	QUARANTENA	posizione nel DAG indefinita: non si sa chi può chiamarlo
nessun entypoint	QUARANTENA	c'è la persona ma non il codice
import del provider	QUARANTENA	aggira il varco, quindi aggira il budget
chiave API nel sorgente	QUARANTENA	le chiavi stanno solo nel varco
venv non provisionato	IN ATTESA	conforme, ma senza dipendenze non parte

CONFORMATO

Ciò che manca ed è deducibile viene dedotto. L'agente gira.

IN ATTESA

Conforme ma non eseguibile.
Manca un presupposto, non una regola.

QUARANTENA

Trattenuto, col motivo scritto.
Non esiste il forse.

Ogni riga della colonna centrale è una decisione presa una volta e applicata sempre, non un giudizio caso per caso.

Conformato, in attesa, quarantena. Ogni riga è una decisione presa una volta e applicata sempre, non un giudizio caso per caso.

Il default è il più prudente

Quando manca la posizione topologica e va dedotta, il valore assegnato è quello di foglia. È la posizione più conservativa: una foglia non delega, quindi non può innescare ramificazioni inattese. Promuoverla a coordinatore resta una decisione da prendere esplicitamente.

Anche questo è un principio con un nome e una data. Saltzer e Schroeder lo chiamano *fail-safe defaults*: le decisioni di accesso devono basarsi sul permesso, non sull'esclusione, e in caso di dubbio il sistema deve negare.¹⁰

La conseguenza pratica è che un agente scritto in fretta e senza attenzione fa poco, invece di fare molto in modo imprevedibile. È il comportamento che vuoi in un sistema dove il costo di un errore non è locale.

Il fallimento silenzioso è la categoria peggiore

Una delle righe della tabella merita di essere spiegata, perché rappresenta un'intera classe di problemi. Due tag di protocollo sono incompatibili fra loro: quello che fa emettere la risposta a pezzi e quello che permette all'agente di interrompersi e chiedere un chiarimento.

Presi insieme, il chiarimento finirebbe dentro il flusso di testo come una frase qualsiasi, e nessuno lo intercetterebbe come richiesta. Il sistema non darebbe errore. Continuerebbe a funzionare, restituendo a chi ha chiesto una domanda travestita da risposta.

Questa è la ragione per cui la combinazione va in quarantena invece di essere accettata con un avvertimento. Un guasto rumoroso costa un'ora di lavoro. Un guasto silenzioso costa la fiducia in tutti gli output del sistema, perché una volta che sai che può succedere non sai più quali risposte guardare.

Normalizzare senza sovrascrivere

Il componente che porta ciò che esiste allo standard ha un vincolo preciso: non sovrascrive niente. Aggiunge quello che manca, non corregge quello che c'è. La distinzione è quella fra un sistema che aiuta e un sistema che decide al posto tuo, e la seconda è insopportabile per chi lavora.

Deduzione senza modello

Quando il dominio di un agente non è dichiarato, viene classificato leggendo il testo che lo descrive. La cosa interessante è come: con un conteggio di parole chiave, non con un modello linguistico.

La scelta è deliberata e la ragione è la ispezionabilità. Un classificatore lessicale non sbaglia mai a caso: se in strada male, si vede quale parola l'ha fatto, e si corregge. Un modello che classifica bene il novantacinque per cento delle volte introduce un cinque per cento di errori non spiegabili, dentro un componente che deve essere prevedibile per definizione.

C'è anche una regola sul dubbio: in caso di parità fra due domini, il risultato è il dominio generico di ripiego. Preferire "non so" a una scelta arbitraria è la stessa logica del default prudente, applicata alla classificazione.

Il criterio generale è utile ben oltre questo caso. Un modello linguistico serve dove il compito richiede comprensione e dove un errore occasionale è tollerabile. Nel percorso che decide se un componente può girare, nessuna delle due condizioni è soddisfatta.

IL PROBLEMA	Le regole scritte in un documento non vengono applicate. I sistemi accumulano componenti che girano senza che nessuno sappia in che stato siano, e i guasti silenziosi rendono inaffidabile tutto l'output.
LA DECISIONE	Un controllo obbligatorio all'ingresso con tre esiti e nessun quarto, che deduce ciò che è deducibile, trattiene il resto col motivo scritto, e in caso di dubbio assegna sempre meno capacità.
COSA COSTA	Rallenta la prototipazione: ogni prova passa dal controllo anche quando sai già che il codice è provvisorio. Le regole del gating diventano esse stesse un'interfaccia, e cambiarle rompe agenti che prima passavano.

CAPITOLO 6

Il ciclo di una richiesta

Sei controlli in sequenza. L'ordine non è casuale, e ognuno esiste per una ragione che si può dire in una frase.

Questo capitolo è il più concreto della guida, perché descrive cosa succede davvero fra il momento in cui arriva una richiesta e il momento in cui torna una risposta. Vale la pena leggerlo con una domanda in testa: per ciascun controllo, cosa succederebbe se non ci fosse.

GOVERNARE UNO SWARM DI AGENTI · FIG. 3

Cosa succede a una richiesta

- 01 Esiste ed è conforme?**
Il record viene letto dal disco, non da una cache autorevole.
`404 inesistente` `409 quarantena`
- 02 La profondità è nei limiti?**
Il kernel limita già la ricorsione, ma è codice che gira dentro l'agente. Qui siamo sul confine di processo, che non si dimentica.
`400 oltre il limite`
- 03 Il chiamante ha la clearance?**
Applicata anche qui, non solo nel router: un agente potrebbe chiamare l'endpoint a mano e saltare il routing.
`403 clearance negata`
- 04 La risposta esiste già?**
Answer-cache: stessa domanda, stesso agente, stesso punto di conversazione. Salta l'intero ragionamento.
`nessun processo, nessun token`
- 05 Esecuzione nel suo ambiente**
Sottoprocesso controllato: nessuna chiave del provider, timeout finito, payload su stdin.
`402 guardia di spesa` `504 timeout`
- 06 Traduzione dell'esito**
Successo, chiarimento o errore.
`200 anche il chiarimento`

PERCHÉ QUEST'ORDINE

I controlli che costano poco stanno prima di quelli che costano molto.

L'autorizzazione sta prima della cache.

Chi non può chiedere non riceve risposta, nemmeno una già pronta.

IL CHIARIMENTO NON È UN GUASTO

Una domanda di ritorno è una risposta valida. Marcarla 5xx farebbe scattare i retry di qualsiasi client ragionevole, e il sistema ripeterebbe la domanda invece di aspettare.

Cosa succede a una richiesta. I controlli economici stanno prima di quelli costosi, e l'autorizzazione sta prima della cache.

Il criterio dell'ordine

Due principi governano la sequenza. Il primo è economico: i controlli che costano poco stanno prima di quelli che costano molto. Leggere un record dal disco costa meno che avviare un processo, quindi si legge prima. È ovvio, e va detto perché l'ovvio si dimentica quando si aggiungono controlli nel tempo, ognuno nel punto in cui è venuto in mente.

Il secondo non è economico ed è più importante: l'autorizzazione sta prima della cache. Se la cache venisse consultata per prima, un chiamante senza diritto riceverebbe una risposta già pronta. Sarebbe gratis, sarebbe veloce, e sarebbe una violazione. La stessa regola vale nel caching HTTP, dove una cache condivisa non può servire risposte a richieste autenticate se non quando è esplicitamente consentito.¹⁶

I sei controlli

Esiste ed è conforme. Il record dell'agente viene letto dal disco, non da una cache che potrebbe essere disallineata. È il capitolo 3 applicato: la fonte di verità è l'osservazione. Se l'agente non esiste, la risposta lo dice; se è in quarantena, la risposta include il motivo, così chi la riceve sa cosa sistemare.

La profondità è nei limiti. Il kernel comune limita già la ricorsione, ma quello è codice che gira dentro l'agente, e ciò che gira dentro può essere aggirato da chi scrive l'agente. Il controllo sul confine di processo non può essere dimenticato, perché non appartiene a nessuno che possa dimenticarlo.

È il pattern che in sicurezza si chiama difesa in profondità: lo stesso controllo in due punti indipendenti, dove il secondo non presuppone la correttezza del primo. Nella pratica il limite è tre, e la cifra conta meno del fatto che sia dichiarata in un posto solo.

Il chiamante ha la clearance. La regola del capitolo 4 viene applicata di nuovo qui, e non solo dentro il meccanismo che instrada le richieste. La ragione è concreta: un agente potrebbe chiamare l'endpoint direttamente e saltare l'instradamento. Se la regola visse solo nel router, esisterebbe un percorso che la aggira, e un percorso che la aggira è tutto quello che serve.

La risposta esiste già. Qui interviene la cache di primo livello, e qui si vede perché sta al quarto posto e non al primo. A questo punto sappiamo che l'agente esiste, che la catena è nei limiti e che il chiamante ha diritto di chiedere. Solo allora una risposta già calcolata è servibile.

Esecuzione nel suo ambiente. L'agente parte come sottoprocesso con un ambiente controllato: nessuna credenziale del fornitore di modelli, un timeout finito, i dati in ingresso passati sullo standard input. Le tre cose insieme definiscono il perimetro di ciò che quel processo può fare, e il perimetro è deciso da chi lo avvia, non da chi lo ha scritto.

Traduzione dell'esito. Il risultato è un successo, un chiarimento o un errore, e va tradotto in una forma che chi ha chiamato possa interpretare senza leggere il testo.

Perché un chiarimento non è un errore

Questo è il punto del capitolo che ha le conseguenze meno intuitive. Un agente che si interrompe per chiedere un dato mancante ha prodotto una risposta valida: la risposta corretta a una domanda incompleta è una domanda.

Se quella risposta venisse marcata come guasto del server, qualsiasi client scritto ragionevolmente farebbe la cosa che si fa con i guasti temporanei: riprovarebbe. E riprovando otterrebbe la stessa richiesta di chiarimento, che riprovarebbe ancora. Il sistema entrerebbe in un ciclo di tentativi su una situazione che non è un guasto e non si risolve da sola.

La lezione generale è che i codici di stato non descrivono come è andata a chi ha scritto il codice: prescrivono cosa deve fare chi li riceve. Sceglierli in base al sentimento invece che all'azione che innescano è uno dei modi più comuni di costruire cicli involontari.

LA DOMANDA DA FARSI SU OGNI CONTROLLO

Se questo controllo non ci fosse, quale comportamento diventerebbe possibile, e me ne accorgerei? Se la risposta alla seconda parte è no, il controllo non è opzionale a prescindere da quanto sia improbabile il caso.

IL PROBLEMA	I controlli aggiunti nel tempo si dispongono nell'ordine in cui sono venuti in mente. Un'autorizzazione valutata dopo una cache non è un'autorizzazione, e un codice di stato scelto male innesca cicli di tentativi.
LA DECISIONE	Una sequenza dichiarata: prima i controlli economici, l'autorizzazione sempre prima della cache, il limite di profondità ripetuto sul confine di processo, e una traduzione dell'esito che distingue il chiarimento dal guasto.
COSA COSTA	Ogni richiesta paga tutti i controlli, anche quelle che non ne avrebbero bisogno. La sequenza è rigida: inserire un controllo nuovo obbliga a ragionare su tutta la catena invece che sul punto in cui serve.

CAPITOLO 7

Agenti e capacità

Un agente ha una persona, un ambiente, un budget, una posizione nel grafo e un processo che parte. È la struttura giusta per qualcosa che ragiona, ed è sovradimensionata per qualcosa che esegue.

La domanda di questo capitolo è di granularità: quando un'unità di lavoro merita un'entità autonoma con identità propria, e quando basta una funzione che si chiama per nome. È la stessa domanda che si pone chi decide quanto grande debba essere un servizio in un'architettura a servizi, e la risposta ha la stessa forma.

Il criterio

Scrivere un pezzo di testo o generare un documento sono capacità, non ruoli. Non hanno un punto di vista, non prendono decisioni che vale la pena attribuire a qualcuno, non hanno bisogno di un budget separato perché non decidono quanto lavorare.

Una capacità è quindi un'unità invocabile leggera ospitata dal control-plane: niente persona, niente ambiente dedicato, niente posizione nel grafo. Si chiama per nome, con dei parametri, e risponde.

GOVERNARE UNO SWARM DI AGENTI · FIG. 6

Agente o capacità

RAGIONA	ESEGUE
Agente	Capacità
Persona	no
Ambiente proprio	no
Budget proprio	no
Posizione nel DAG	no
Processo che parte	si
Struttura giusta per un ruolo.	Struttura giusta per una funzione.

può stare nel control-plane ⇔ per uscire serve al massimo il varco LLM

Il control-plane è in gabbia di rete. Una capacità che genera testo passando dal varco può essere ospitata lì. Una che chiama un servizio esterno, manda una email o vuole un browser non può, e resta a catalogo marcata non invocabile col motivo: «non c'è» e «non può esserci qui» sono risposte diverse.

Il criterio non è la complessità del compito. È se serve un'identità con un budget, o basta una funzione invocabile.

Dare a tutto un'identità moltiplica le entità da governare; darne a niente rimette la governance dentro il codice.

Agente o capacità. Il criterio non è la complessità del compito, ma se serve un'identità con un budget o basta una funzione invocabile.

Il costo di sbagliare in entrambe le direzioni

Dare un'identità a tutto moltiplica le entità da governare. Ogni agente ha una posizione da decidere, un budget da assegnare, un ambiente da mantenere. Dieci agenti che avrebbero potuto essere tre agenti e sette funzioni sono dieci volte il lavoro di governance per lo stesso risultato. È l'antipattern che nella letteratura sui servizi viene chiamato dei nanoservizi: frammentazione che produce costo di coordinamento senza produrre autonomia.

Sbagliare nella direzione opposta costa in modo diverso. Se una cosa che decide viene implementata come funzione, la sua logica finisce dentro chi la chiama, e con essa il suo budget, il suo comportamento in caso di errore e la sua posizione nel grafo. La governance non scompare: rientra nel codice, che è precisamente il posto da cui il capitolo 2 la voleva togliere.

Il criterio utile è quello di Parnas, applicato a un livello diverso da quello per cui fu scritto: ogni modulo dovrebbe nascondere una decisione che può cambiare.¹⁷ Se l'unità di lavoro incapsula una decisione che può cambiare indipendentemente dalle altre, merita un confine proprio. Se esegue e basta, il confine è costo puro.

Il vincolo di rete decide dove può stare

C'è un secondo criterio, indipendente dal primo, e riguarda dove una capacità può essere ospitata. Il control-plane sta in una rete isolata, senza accesso a internet, con una sola via di uscita: il varco verso i modelli linguistici.

Da qui la regola: può stare lì dentro solo ciò che, per fare il proprio lavoro, non ha bisogno di uscire da nessun'altra parte. Una capacità che genera testo passando dal varco può essere ospitata. Una che deve chiamare un servizio esterno, mandare una email o avviare un browser non può, e non è un limite da aggirare: è l'isolamento che funziona come previsto.

Il dettaglio che rende la scelta buona invece che solo restrittiva è come vengono trattate le capacità che non possono stare lì. Restano a catalogo, marcate come non invocabili, col motivo. Perché «non c'è» e «non può esserci qui» sono risposte diverse, e chi le riceve deve poterle distinguere: la prima è un lavoro da fare, la seconda è una decisione già presa.

Perché limitare l'agire è un principio, non una precauzione

La classifica OWASP dei rischi per le applicazioni basate su modelli linguistici dedica una voce a quella che chiama agency eccessiva, e la scompone in tre cause: funzionalità eccessive, cioè strumenti raggiungibili oltre l'ambito del compito; permessi eccessivi, cioè strumenti che operano con privilegi più ampi del necessario; e autonomia eccessiva, cioè azioni ad alto impatto che procedono senza una persona nel circuito.¹⁸

Le prime due sono esattamente ciò che il vincolo di rete affronta. Un componente che non può raggiungere la rete esterna non ha funzionalità oltre il proprio ambito, non perché gli sia stato detto di non usarle, ma perché non ci sono.

È la forma più antica di questo principio: il minimo privilegio, che prescrive che ogni programma operi con l'insieme minimo di privilegi necessari a completare il proprio lavoro.¹⁰ La versione moderna, nei sistemi a capacità, aggiunge una sfumatura utile: il privilegio non è un attributo di chi agisce, è un oggetto che si possiede o non si possiede.¹⁹ Se non ti è stata data la capacità di uscire in rete, non esiste una configurazione sbagliata che te la restituisca.

IL PROBLEMA	Trattare ogni unità di lavoro come un agente moltiplica le entità da governare senza produrre autonomia. Tratarle tutte come funzioni rimette la governance dentro il codice di chi le chiama.
LA DECISIONE	Identità e budget solo a ciò che decide. Tutto il resto è una funzione invocabile per nome. Dove può essere ospitata lo decide un vincolo di rete esplicito, e ciò che non può stare dentro resta a catalogo col motivo.
COSA COSTA	Il confine fra ruolo e funzione non è netto, e alcune unità cambiano natura nel tempo. Promuovere una capacità ad agente è un lavoro di riscrittura, non un cambio di etichetta.

CAPITOLO 8

La spesa come vincolo di governance

Due livelli di cache tolgono lavoro al sistema. Nessuno dei due toglie mai un permesso.

La distinzione del titolo è l'unica cosa che conta di questo capitolo. Una guardia di spesa che blocca un agente non è una misura di costo: è una decisione di governance. Qualcuno ha stabilito che quell'agente, in questo momento, non deve lavorare. Il fatto che una risposta costerebbe zero non cambia la decisione, la aggira.

Due livelli, due cose diverse

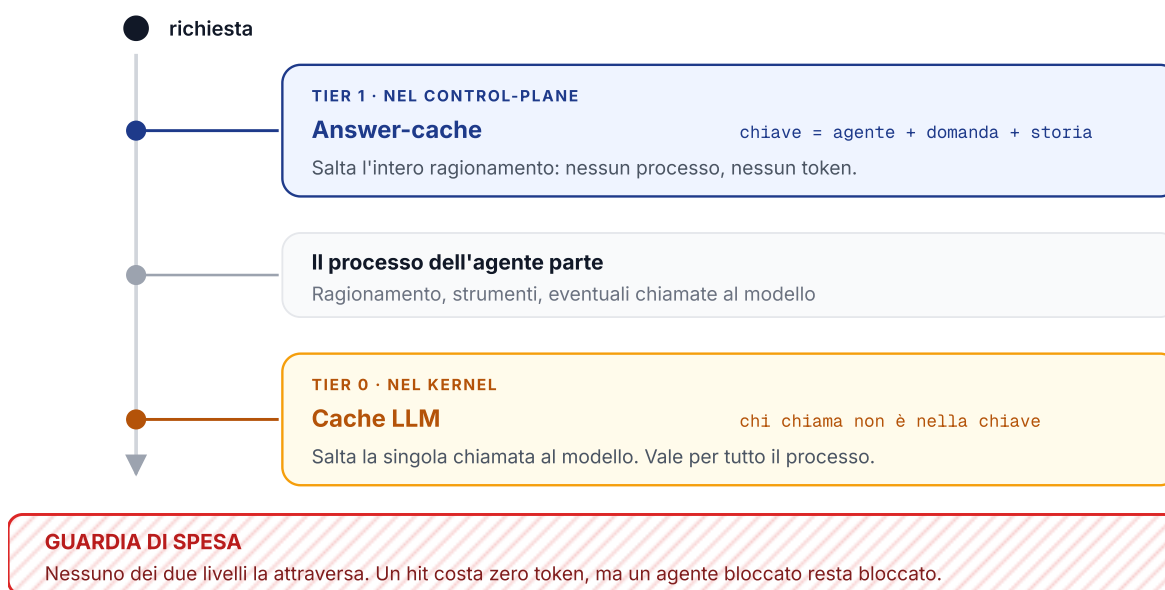
Il primo livello sta nel control-plane e intercetta la richiesta prima che qualsiasi cosa parta. Se la stessa domanda è già stata posta allo stesso agente nello stesso punto di conversazione, la risposta esiste: nessun processo si avvia, nessun ragionamento viene rifatto.

Il secondo livello sta nel kernel comune e interviene molto più in basso, sulla singola chiamata al modello. Si attiva alla sola importazione, quindi vale per l'intero processo e anche per chi poi chiama il varco a mano. È il capitolo 3 applicato ai costi: una funzionalità trasversale che dipendesse dalla diligenza di chi scrive l'agente sarebbe una funzionalità che hai a metà.

GOVERNARE UNO SWARM DI AGENTI · FIG. 7

I due livelli di cache

IL PERCORSO DI UNA RICHIESTA



Le cache tolgono lavoro. Non tolgono mai un permesso: servire dalla cache a chi è bloccato significherebbe aggirare il blocco.

I due livelli di cache. Intercettano in punti diversi del percorso, e la guardia di spesa non è attraversata da nessuno dei due.

Cosa entra nella chiave, e perché

Una cache è tanto corretta quanto lo è la sua chiave. Se due situazioni diverse producono la stessa chiave, la cache restituisce la risposta sbagliata, e lo fa con l'aria di funzionare.

Nel primo livello la storia della conversazione entra nella chiave. Senza, in una chat con memoria la stessa domanda posta in due momenti diversi tornerebbe con la risposta del contesto sbagliato. È un errore invisibile a chi guarda i log e ovvio a chi legge la conversazione, cioè la combinazione peggiore.

Il problema è vecchio e ha una formalizzazione precisa nel caching del web. La chiave primaria di una cache HTTP è composta almeno dal metodo e dall'indirizzo della risorsa; quando la risposta dipende anche da altro, il server lo dichiara, e la cache non può riusare una risposta memorizzata se quei campi non corrispondono.¹⁶ La storia della conversazione è esattamente uno di quei campi: fa parte di ciò che ha determinato la risposta, quindi fa parte della chiave.

Nel secondo livello, invece, l'identità di chi chiama non entra nella chiave. È una scelta deliberata con una conseguenza dichiarata: il primo che paga un prompt lo paga per tutti. Ha senso perché a quel livello la risposta dipende dal prompt e non da chi lo ha formulato, e perché il beneficio della condivisione è precisamente lo scopo del livello.

Cosa non va mai in cache

Quattro categorie restano fuori, e ognuna per una ragione diversa. Una richiesta di chiarimento non è una risposta ma una domanda: servirla dalla cache significherebbe richiedere un chiarimento già dato. Una risposta emessa a pezzi è una sequenza nel tempo, non un valore. Se l'agente ha modificato uno stato, la stessa domanda la seconda volta ha un effetto diverso, e servire la risposta memorizzata cancella quell'effetto. E infine l'agente può sempre dichiarare che una risposta non è ripetibile: chi conosce il proprio dominio ha l'ultima parola.

C'è un quinto caso che vale la pena isolare, perché è il più facile da sbagliare: quando un agente cambia, le risposte memorizzate prima del cambiamento non sono più sue. La soluzione adottata qui non è mettere la versione nella chiave, ma invalidare tutto ciò che riguarda quel agente a ogni suo rilascio. La differenza pratica è che nel primo caso le risposte vecchie restano a occupare spazio per sempre, nel secondo spariscono.

nessuna cache attraversa la guardia di spesa

Un risultato dalla cache costa zero, ma un agente bloccato resta bloccato. Servire una risposta memorizzata a chi è stato fermato significherebbe che il blocco vale solo per le domande nuove, cioè che non vale.

IL PROBLEMA	Una chiave di cache incompleta restituisce risposte sbagliate senza dare errore. E una cache messa nel punto sbagliato del percorso trasforma un limite di governance in un limite aggirabile.
LA DECISIONE	Due livelli con chiavi diverse e dichiarate, la storia della conversazione dentro la chiave dove la conversazione conta, l'invalidazione a ogni rilascio, e nessun livello posizionato prima dei controlli di autorizzazione e di spesa.
COSA COSTA	Una chiave che include la storia ha meno riscontri utili di una che non la include: si rinuncia a risparmio in cambio di correttezza. E la condivisione fra chiamanti nel livello basso rende meno preciso attribuire il costo a chi lo ha generato.

CAPITOLO 9

Limiti e trade-off dell'architettura

Questo capitolo esiste perché un documento che presenta solo i vantaggi di una scelta non ti mette in condizione di prenderne una diversa.

La gerarchia è rigida, e la rigidità è il punto

Tutto ciò che rende affidabile l'ordinamento sui rank è la stessa cosa che lo rende scomodo. Non esistono eccezioni, perché un'eccezione riaprirebbe la possibilità di un ciclo e con essa la necessità di controllarli, cioè annullerebbe il vantaggio.

In pratica significa tre cose. Due agenti dello stesso livello che dovrebbero confrontarsi non possono farlo: devono passare da chi sta sopra o attraverso un ponte, con un giro in più e una perdita di contesto a ogni passaggio. Cambiare la posizione di un agente non è una modifica locale, perché tocca tutte le relazioni che lo riguardano. E la gerarchia va decisa presto, quando ancora non sai come si distribuirà davvero il lavoro.

Il gating rallenta chi esplora

Un controllo obbligatorio all'ingresso è prezioso quando il sistema è in esercizio e fastidioso quando stai provando. Nella fase in cui vuoi capire se un'idea funziona, sapere che ogni tentativo passa comunque dalla verifica cambia il ritmo del lavoro, e il rischio concreto è che qualcuno costruisca un percorso alternativo per andare più veloce.

La mitigazione è il principio del capitolo 5, la deduzione automatica di ciò che manca, e riduce il problema senza eliminarlo. Se il tuo lavoro è prevalentemente esplorativo, il conto che paghi è più alto del beneficio che ricevi.

Cosa la topologia non risolve

Va detto chiaramente. L'ordinamento garantisce che il sistema termini e che nessuno chiami chi non deve. Non dice niente sulla qualità di ciò che viene passato lungo la catena.

La degradazione del contesto e la propagazione dell'errore, due dei quattro problemi del capitolo 1, restano interamente aperti. Un sistema perfettamente aciclico può produrre, con ordine impeccabile, una risposta sbagliata costruita su una premessa sbagliata.

L'argomento contrario, per intero

Esiste una posizione pubblica e argomentata secondo cui i sistemi multi-agente vanno evitati. Viene da chi costruisce agenti di programmazione in produzione, e merita di essere riportata senza addolcirla, perché attacca la premessa stessa di questa guida.

La tesi si regge su due principi. Il primo: bisogna condividere il contesto, e condividere le tracce complete di esecuzione degli agenti, non i singoli messaggi. Il secondo: le azioni portano con sé decisioni implicite, e decisioni implicite in conflitto producono risultati sbagliati.⁴

L'esempio che accompagna l'argomento è concreto. Due sotto-agenti lavorano in parallelo a parti diverse dello stesso compito. Ciascuno, per procedere, assume qualcosa che l'altro non può vedere: uno costruisce nello stile A, l'altro nello stile B. Nessuno dei due sbaglia il proprio pezzo. Il risultato combinato è incoerente, e chi ha delegato deve riconciliare due output costruiti su premesse diverse, cosa che spesso costa più che rifare.

La raccomandazione conseguente è di usare per default un agente singolo, lineare, e di affrontare il problema del contesto che cresce con un modello che comprime la conversazione in decisioni e dettagli chiave, invece che con la ramificazione.

Perché regge, e dove

L'argomento è corretto, e non nel senso diplomatico. Il fallimento che descrive è reale, è frequente, ed è esattamente il tipo di fallimento che la ricerca di Berkeley classifica sotto disallineamento fra agenti.³ Chi dice che nessun ordinamento topologico lo previene ha ragione: l'ordinamento riguarda chi può chiamare chi, non cosa si sanno l'un l'altro.

Anche la documentazione di Anthropic sul proprio sistema multi-agente, che è un sistema multi-agente e quindi non è una fonte ostile, dichiara lo stesso limite: i domini che richiedono contesto condiviso fra tutti i partecipanti, o che hanno molte dipendenze fra i sotto-compiti, oggi non sono adatti a essere divisi fra agenti.²

Dove la divisione conviene comunque

La distinzione utile non è fra un agente e molti, ma fra compiti in cui i sotto-problemi sono indipendenti e compiti in cui non lo sono. Cercare informazioni da fonti diverse si divide bene, perché ogni ricerca è autonoma e i risultati si sommano. Scrivere codice su una base condivisa si divide male, perché ogni scelta vincola le successive.

Una regola pratica che tiene conto dell'argomento contrario senza rinunciare del tutto alla divisione: mantenere singolo ciò che scrive, e dividere ciò che legge. Più agenti che raccolgono, analizzano e propongono, e un solo punto in cui le decisioni vengono prese e applicate. Chi legge in parallelo non genera decisioni in conflitto, perché non ne prende.

Quando un'architettura più piatta è preferibile

Tre casi, senza sfumature. Se hai meno di cinque agenti e nessuno ne chiama un altro, tutto questo apparato è sovrastruttura: ti serve un elenco e un modo di avviarli. Se il tuo dominio richiede negoziazione continua fra pari, l'ordinamento ti costringerà a fingere gerarchie che non esistono, e le gerarchie finte producono ponti che diventano colli di bottiglia. Se stai ancora capendo quali agenti ti servono, congelare la topologia significa congelare un'ipotesi.

IL RISCHIO DEL PUNTO UNICO

Vale la pena ripeterlo qui, perché è il costo strutturale più grande di tutta l'architettura. Un control-plane concentra coerenza e concentra fragilità nello stesso punto. Se cade, non gira niente. Se la logica di autorizzazione ha un difetto, il difetto vale per tutto il sistema contemporaneamente. È una scelta che si fa a occhi aperti, non un dettaglio implementativo.

IL PROBLEMA	Ogni garanzia strutturale si paga in flessibilità, e le garanzie di questa architettura riguardano la terminazione e l'autorizzazione, non la coerenza di ciò che gli agenti si passano.
LA DECISIONE	Accettare la rigidità dove il beneficio è una proprietà dimostrabile, e riconoscere che il problema del contesto condiviso resta aperto: dividere ciò che legge, tenere singolo ciò che scrive.
COSA COSTA	Prototipazione più lenta, pari che non collaborano, una topologia decisa prima di sapere se è quella giusta, e un punto unico che concentra il rischio insieme al controllo.

CAPITOLO 10

Le decisioni che devi prendere tu

Otto scelte, con il criterio per decidere e il segnale che ti dice se hai deciso male. Nessuna dipende dal linguaggio o dalla libreria che usi.

1 L'assenza di cicli è una proprietà o un controllo?

CRITERIO

Se puoi ordinare i tuoi agenti in livelli e ammettere la delega in una sola direzione, l'aciclicità diventa gratuita. Se non puoi, ti serve un rilevamento a runtime, e allora ti serve anche lo stato condiviso per farlo funzionare.

HAI DECISO MALE SE

ti trovi ad alzare il limite di profondità per non tagliare lavoro legittimo.

2 Esiste più di un modo di avviare un agente?

CRITERIO

Conta i percorsi. Se sono più di uno, le tue regole valgono su un sottoinsieme, e prima o poi qualcuno userà l'altro. Un punto di ingresso unico è una preconditione, non una raffinatezza.

HAI DECISO MALE SE

esiste uno script che avvia un agente saltando i controlli, e qualcuno lo usa perché è più veloce.

3 L'inventario è dichiarato o derivato?

CRITERIO

Se esiste una lista che qualcuno deve ricordarsi di aggiornare, quella lista divergerà. Chiediti se puoi derivarla dall'osservazione. Se non puoi, metti un controllo automatico che segnali la divergenza.

HAI DECISO MALE SE

hai già trovato almeno una volta un componente che esisteva e non era in elenco.

4 Dove sta scritta la persona di un agente?

CRITERIO

In un posto solo, e il codice la importa. Se compare in due posti, cambierà in uno solo, e avrai due comportamenti a seconda del percorso.

HAI DECISO MALE SE

per capire come si comporta un agente devi leggere due file e confrontarli.

5 Cosa succede a un componente non conforme?

CRITERIO

Tre esiti e nessun quarto: conformato con la deduzione scritta, in attesa perché manca un presupposto, trattenuto col motivo. Lo stato "gira ma non sappiamo in che condizioni" va eliminato per primo.

HAI DECISO MALE SE

quando qualcosa viene rifiutato, chi lo riceve deve indovinare cosa sistemare.

6 In che direzione si risolve il dubbio?

CRITERIO

Sempre verso meno capacità. Il valore assegnato quando manca un'informazione deve essere quello che limita, non quello che abilita. Vale per la posizione nel grafo, per i permessi e per la classificazione.

HAI DECISO MALE SE

un componente scritto in fretta ottiene per default più diritti di quelli che gli avresti dato pensandoci.

7 Cosa entra nella chiave della cache?

CRITERIO

Tutto ciò che ha concorso a determinare la risposta. Se la conversazione influenza l'output, la conversazione è nella chiave. Se il componente cambia, le risposte vecchie non sono più sue: o versioni la chiave, o invalidi al rilascio.

HAI DECISO MALE SE

ti è già capitato di ricevere una risposta corretta per una domanda che non avevi fatto.

8 Questa unità di lavoro merita un'identità?

CRITERIO

Solo se incapsula una decisione che può cambiare da sola, e se ha senso darle un budget. Se esegue e basta, è una funzione: darle identità moltiplica il lavoro di governance senza produrre autonomia.

HAI DECISO MALE SE

hai agenti a cui non sapresti assegnare un budget separato, o funzioni di cui discuti il comportamento come se fossero persone.

UNA DOMANDA CHE VALE PER TUTTE E OTTO

Se questa regola venisse violata stanotte, me ne accorgerei domani? Dove la risposta è no, la regola non può stare in un documento: deve stare in un punto di passaggio obbligato, oppure deve essere resa inesprimibile.

CAPITOLO 11

Fonti e letture

Tutte consultate il 4 agosto 2026, raggruppate per il problema che aiutano a risolvere.

COME SI ORCHESTRANO PIÙ AGENTI

- 1 **Building effective agents**, Anthropic, 19 dicembre 2024.
anthropic.com/engineering/building-effective-agents
Il punto di partenza: i pattern di base, e l'avvertimento che la complessità va aggiunta solo quando migliora il risultato in modo dimostrabile.
- 2 **How we built our multi-agent research system**, Anthropic, giugno 2025.
anthropic.com/engineering/multi-agent-research-system
I moltiplicatori di consumo, e la dichiarazione esplicita dei domini in cui la divisione fra agenti oggi non funziona.
- 6 **Multi-agent architectures**, documentazione LangGraph, LangChain.
langchain-ai.github.io/langgraph
La tassonomia esplicita di rete, supervisor e gerarchico, e il passaggio del controllo fra agenti.
- 7 B. Hayes-Roth, **A blackboard architecture for control**, Artificial Intelligence 26(3), 1985.
La fonte originale del pattern a lavagna, dove il problema del controllo è affrontato più esplicitamente che nelle riscritture recenti.

COME FALLISCONO

- 3 M. Cemri et al., **Why Do Multi-Agent LLM Systems Fail?**, arXiv:2503.13657, 17 marzo 2025.
arxiv.org/abs/2503.13657
Quattordici modi di fallire in tre categorie, da oltre milleseicento tracce su sette framework. Molti fallimenti vengono dal progetto, non dal modello.
- 4 W. Yan, **Don't Build Multi-Agents**, Cognition, giugno 2025.
cognition.ai/blog/dont-build-multi-agents
L'argomento contrario, riportato per intero nel capitolo 9. Da leggere soprattutto se hai deciso di costruire un sistema multi-agente.
- 5 N. F. Liu et al., **Lost in the Middle: How Language Models Use Long Contexts**, arXiv:2307.03172, 6 luglio 2023.
arxiv.org/abs/2307.03172
Perché accumulare contesto lungo la catena non è neutro: l'informazione in mezzo viene usata peggio di quella agli estremi.

PERCHÉ I CICLI SI PREVENGONO INVECE DI RILEVARLI

- 8 A. B. Kahn, **Topological sorting of large networks**, Communications of the ACM 5(11), novembre 1962.
dl.acm.org/doi/10.1145/368996.369025
Il legame formale fra aciclicità e ordinamento lineare dei nodi.
- 9 E. G. Coffman, M. J. Elphick, A. Shoshani, **System Deadlocks**, ACM Computing Surveys 3(2), 1971.
dl.acm.org/doi/10.1145/356586.356588
Le quattro condizioni necessarie del deadlock, fra cui l'attesa circolare. Rende evidente che eliminare una condizione necessaria è diverso dal rilevare il fenomeno.
- 15 J. W. Havender, **Avoiding deadlock in multitasking systems**, IBM Systems Journal 7(2), 1968.
dl.acm.org/doi/10.1147/sj.72.0074
L'ordinamento delle risorse: la stessa mossa dei rank, su un problema diverso, sessant'anni fa.

DOVE SI APPLICANO LE REGOLE

- 10 J. H. Saltzer, M. D. Schroeder, **The Protection of Information in Computer Systems**, Proceedings of the IEEE 63(9), 1975.
`cs.virginia.edu/~evans/cs551/saltzer`
Da qui vengono la mediazione completa, i default sicuri e il minimo privilegio. Se leggi una sola fonte di questo elenco, leggi questa.
- 11 **Admission Control in Kubernetes**, documentazione Kubernetes.
`kubernetes.io/docs/reference/access-authn-authz/admission-controllers`
Il gating del capitolo 5 in un altro dominio: controlli che modificano o rifiutano prima della persistenza, e il rifiuto di uno che rifiuta tutta la richiesta.
- 12 **Open Policy Agent**, documentazione del progetto, Cloud Native Computing Foundation.
`openpolicyagent.org/docs`
Le regole di autorizzazione come codice valutabile e versionato, fuori dall'applicazione.
- 13 **Controllers**, documentazione Kubernetes.
`kubernetes.io/docs/concepts/architecture/controller`
Stato desiderato e stato osservato come campi distinti, e il ciclo che ne riduce la distanza. È il modello a cui la scoperta da disco rinuncia.
- 19 J. B. Dennis, E. C. Van Horn, **Programming Semantics for Multiprogrammed Computations**, Communications of the ACM 9(3), 1966.
`dl.acm.org/doi/10.1145/365230.365252`
L'origine della protezione a capacità: il privilegio come oggetto che si possiede, non come attributo di chi agisce.

ISOLAMENTO, GRANULARITÀ, COSTI

- 14 **Config**, The Twelve-Factor App.
`12factor.net/config`
La configurazione, e in particolare i segreti, nell'ambiente e mai nel codice.
- 16 R. Fielding, M. Nottingham, J. Reschke, **RFC 9111: HTTP Caching**, IETF, giugno 2022.
`rfc-editor.org/rfc/rfc9111.html`
Composizione della chiave, chiavi secondarie, e il divieto per le cache condivise di servire risposte a richieste autenticate. Il capitolo 8 ne è una rilettura.
- 17 D. L. Parnas, **On the Criteria To Be Used in Decomposing Systems into Modules**, Communications of the ACM 15(12), dicembre 1972.
`dl.acm.org/doi/10.1145/361598.361623`
Ogni modulo nasconde una decisione che può cambiare. Ancora il miglior criterio per decidere dove passa un confine.
- 18 **OWASP Top 10 for LLM Applications 2025**, voce LLM06 Excessive Agency, OWASP Foundation.
`owasp.org/www-project-top-10-for-large-language-model-applications`
Funzionalità, permessi e autonomia eccessivi come tre cause distinte da trattare separatamente.

Grazie per aver letto

Se questa guida ti è stata utile, girala a qualcuno che sta mettendo in fila più di un agente e non ha ancora deciso come li tiene insieme. È il momento in cui serve.

Omar Bortolato

Chief AI Officer · Imprenditore · Speaker

omarbortolato.it · linkedin.com/in/omarbortolato